

Wie man eine PHP-Slot-Engine von 5.000 auf über 300.000 Spins pro Sekunde beschleunigt und Millionen an Infrastrukturkosten spart

Autor: Andrii Starkov (Senior/Staff Software Engineer)

Projekt-Repository: [GitHub Repository](#)

Professionelle Kontakte: [LinkedIn Profile](#)

Inhaltsverzeichnis

1. **Einleitung** (Teststand, Basisspiel, Links zu GitHub/LinkedIn)
2. **Analyse von technischen Schulden im iGaming** (Berechnungsspezifika, Legacy, Monte Carlo als Bottleneck)
3. **Architektur-Entscheidungen für Ultra-Performance in PHP** (OOP eliminieren, flache Arrays, Short-Circuit, Mikro-Optimierungen)
4. **Benchmark-Ergebnisse** (Geschwindigkeitsvergleich: 5k vs. 304k Spins/Sek., Business-Nutzen)
5. **Die Grenze der Vernunft: Dunkle Magie der Optimierung** (Bitmasken, prozedurales Chaos, vorausberechnete Gewinnkarten)
6. **Migration zu Go (Golang) in 3 Wochen** (Strikte Typisierung, Slice-Optimierung, Millionen von RPS)
7. **Fazit** (PHP-Mythen, „Mechanical Sympathy“ für Hardware)

1. Einleitung

Hallo! Ich bin Senior/Staff Software Engineer mit mehr als 11 Jahren Erfahrung in Backend-/Frontend-Entwicklung, Systemdesign und Enterprise-Architektur. Mein Profil umfasst hochbelastete Systeme mit schwerem Legacy-Code, komplexer Geschäftslogik und das Refactoring von Monolithen mit zehnjähriger Geschichte im laufenden Betrieb, ohne Produktionsstopp.

Dieser Artikel ist das Ergebnis einer tiefen Analyse von Architekturproblemen in Hunderten von realen Spielen und einer Codebasis, die sich über ein Jahrzehnt angesammelt hat. Ich habe den Spielkern komplett von Grund auf neu geschrieben und die gesamte Plattforminteraktionslogik nach außen verlagert. Die Engine wurde zu reiner Mathematik.

Nachfolgend finden Sie einen detaillierten technischen Leitfaden dazu, wie Sie PHP zwingen, die Leistung kommerzieller Rechenkerne zu erbringen, warum „schöner“ Code die CPU tötet und wie technische Entscheidungen dem Unternehmen Millionen von Dollar sparen können.

Umgebung und Eigenschaften des Prüfstands

Alle besprochenen Benchmarks und Leistungskennzahlen wurden auf derselben Hardware erfasst:

- **OS:** Debian GNU/Linux 12 (bookworm) — Linux-Kernel 6.1.0-44-amd64 (x86_64)

- **CPU:** 12th Gen Intel(R) Core(TM) i7-1260P (12 Kerne / 16 Threads, bis zu 4.70 GHz)
- **RAM:** 32 GB
- **NVME:** Samsung SSD 990 PRO 1TB
- **Basisspiel:** Für die Reinheit des Experiments wurde ein einfaches Modell gewählt: ein 3x3-Spielfeld, 7 Symbole, 3 Gewinnlinien (Horizontale von links nach rechts). Keine Freispiele, Respins, Bonusrunden oder zusätzliche Mechaniken.

2. Analyse von technischen Schulden im iGaming

Sagen wir es wie es ist: Dem Endkunden ist es völlig egal, wie schön der Code in Ihrem Repository ist. Das Spiel dreht sich, die Slots blinken, das Guthaben stimmt — für ihn funktioniert das System. Man kann auch 20 Jahre lang schlechten Code schreiben, aber für ein Unternehmen ist die interne Architektur entweder Raketentreibstoff oder ein Betonklotz am Bein.

Ich war mit der Codebasis eines großen iGaming-Projekts mit Hunderten von Altspielen konfrontiert, die ein klassisches Museum für technische Schulden darstellte. Hier ist nur ein kleiner Teil davon:

- **Mathematik in float:** Alle Berechnungen wurden in Gleitkommazahlen durchgeführt, was zu klassischen Rundungs- und Präzisionsproblemen führte.
- **Versionierung über massive Switch-Case-Wüsten:** Wurde zur Unterstützung alter Spielregeln verwendet.
- **GodRepository:** Alle Spiele und Tools befanden sich an einem einzigen Ort.
- **Monster-Dateien:** Eine einzelne Datei konnte auf Zehntausende von Codezeilen anwachsen, in denen 10 Funktionen mit wilden Namensmustern koexistierten, die genau dasselbe taten, aber „jede auf ihre Weise“..
- **Vakuum statt Tests und Dokumentation:** Die Codebasis wurde wie die Mythen des antiken Griechenlands weitergegeben — durch Mundpropaganda.

Wenn ein Unternehmen Hunderte von Spielen im Portfolio hat und Zehntausende von Spielern gleichzeitig die Walzen drehen, wird die Serverlast kolossal. Dies ist kein klassisches Web-CRUD. In einer Standard-Webanwendung läuft die CPU meist im Leerlauf und wartet auf eine Antwort von der Datenbank oder dem Netzwerk. Um dort den PHP-Bootstrap-Overhead pro Anfrage zu eliminieren, verwenden wir einfach Tools wie RoadRunner oder Swoole. Aber im iGaming ist der Server kontinuierlich mit reinen, schweren Berechnungen der Spiellogik für jede einzelne Aktion des Spielers beschäftigt.

Und das heizt die CPU stark auf, insbesondere bei Monte-Carlo-Simulationen — der Berechnung von 10.000.000.000 (10 Milliarden) Spins für jedes RTP-Profil. Dies ist ein obligatorischer Schritt zur mathematischen Validierung, Berechnung der Volatilität (Variationskoeffizient) und Überprüfung des RTP, bevor die Software an strenge Zertifizierungslabors (iTech Labs, eCOGRA) gesendet wird.

Wenn Sie eine iGaming-PHP-Engine nach den akademischen Kanons von „sauberem OOP“ schreiben (mit Objekten für jedes Symbol, jede Linie und jedes Spinnergebnis), erstickt PHP an Speicherallokationen und liefert jämmerliche 100–200 Spins pro Sekunde. Aus Erfahrung getestet. Alter Legacy-Code aus der Dinosaurier-Ära lieferte aufgrund schwerer Systemfunktionen (z. B. `array_merge`, `array_flip`), tief verschachtelter Strukturen, die Cache-Misses verursachten, und ständiger Festplatten-I/O nur etwa ~5.000 Iterationen pro Sekunde.

Die nackte Mathematik der Geschäftsverluste

Wenn Sie eine Baseline von 10 Milliarden Iterationen mit der Geschwindigkeit der alten Legacy-Engine von 5.000 Spins/Sek. ausführen müssen, erhalten Sie einen katastrophalen Engpass:

$$10.000.000.000 / (5.000 \times 3.600) \approx 555,6 \text{ Stunden auf einem einzigen Kern}$$

- Auf 10 Kernen parallel — das sind 55,5 Stunden ununterbrochene Arbeit.
- Auf 20 Kernen — 27,7 Stunden.

And das ist nur ein Durchlauf. Dutzende oder Hunderte solcher Zyklen sind pro Spiel erforderlich, um Bindung und Volatilität anzupassen. Infolgedessen zieht sich die Time-to-Market (TTM) über Wochen hinweg, und das Unternehmen verbrennt massive Budgets für die Miete von Multi-Core-Cloud-Instanzen.

3. Architektur-Entscheidungen für Ultra-Performance in PHP

Speichervernichtung: OOP im Hot Path eliminieren

Der Hauptfeind extremer Performance in PHP ist die Speicherallokation und die Garbage Collection (GC). Die Schleifen, die Spielfelder generieren, nach Linien suchen und Auszahlungen berechnen, stellen unseren Hot Path dar, der Milliarden von Aufrufen verarbeitet. In der umfassenderen Architektur sind Entwurfsmuster lebenswichtig. Aber innerhalb des Hot Path muss jede akademische Schönheit sterben. Der Code hier muss primitiv, direkt und roh sein.

- **Objekte gehören weggeschnitten:** Keine Objekte innerhalb der Schleife. Das Instanzieren selbst eines leichtgewichtigen Objekts wie `new WinLine()` oder `new SpinResult()` innerhalb des Hot Path zwingt die CPU, Hunderte von Zyklen für die Strukturallokation und die anschließende GC-Bereinigung aufzuwenden. Nur primitive Datentypen und flache Arrays gehören hierher.
- **Ein bewusster Kompromiss:** Ja, Arrays in PHP sind geordnete Hash-Tabellen (zvals) und allokalieren ebenfalls Speicher, und der Verzicht auf Objekte erschwert das Debugging. Dies ist jedoch eine grundlegende Einschränkung der Sprache, und wir bringen dieses Opfer ganz bewusst. Außerhalb des Hot Path schreiben wir sauberes OOP, aber bevor wir Daten in die Berechnungsschleife einspeisen, reduzieren wir sie auf flache Primitive.
- **Keine Interfaces oder Laufzeitdynamik:** Keine polymorphen Aufrufe, „Strategie“-Muster oder dynamischen Variablen wie `$$variable`. Der PHP 8.5 JIT-Compiler kann Code nicht optimieren, wenn sich Datentypen und Ausführungspfade zur Laufzeit ständig verschieben. Die CPU und der JIT benötigen einen vorhersehbaren Kontrollfluss. Die Reinheit innerhalb des Hot Path muss absolut sein: Wenn `calculateWin()` für die Berechnung von Gewinnen konzipiert ist, tut es nur das. Kein implizites Logging, keine Seiteneffekte.
- **Klonverbot:** Vergessen Sie das Schlüsselwort `clone` und insbesondere tiefes Klonen über schweres `serialize` / `unserialize`. Alle Datenvorbereitungen und Transformationen müssen abgeschlossen sein, bevor die erste Iteration der Schleife ausgelöst wird.

Matrizen flachlegen: Der Krieg gegen Cache-Misses

Zweidimensionale Arrays sind der Standard für die Darstellung eines Spielrasters: `Reels[X][Y]`. Der Zugriff auf `$reels[$x][$y]` zwingt die CPU jedoch zu endlosen Sprüngen über Zeiger, was bei großen Volumina zu L1/L2-Cache-Misses führt. Die CPU stagniert und wartet auf Daten aus dem langsamen RAM.

Lösung: Verbinden Sie alle Walzen von links nach rechts, von oben nach unten, in einem einzigen, flachen 1D-Array. Ein 3x3-Raster wird zu einem Array mit Indizes von 0 to 8.

$$position = rowID \times gameFieldWidth + reelID$$

Nun ist eine Gewinnlinie ganz oben in einem 3x3-Raster eine einfache Indexprüfung von `0, 1, 2`. Die Daten liegen sequentiell im Speicher und sitzen perfekt im L1-Cache der CPU.

Algorithmische Short-Circuit-Optimierung

Bei Milliarden von Operationen führt das Einsparen von selbst 5 % der Ressourcen zu echten Infrastruktureinsparungen. Wir verzichten auf universelle Kalkulatoren zugunsten isolierter Methoden oder Klassen, die auf eine bestimmte Aufgabe zugeschnitten sind. Algorithmen werden genau einmal beim allerersten Spin der Simulation aufgewärmt, wobei Dimensionen zwischengespeichert und Speicherstrukturen vorallokiert werden, die später einfach überschrieben werden.

- **Prüfung von Linien statt Kombinationen:** Wenn ein Spiel 5 Gewinnlinien und 25 potenzielle Symbolkombinationen hat, schleifen wir strikt über die 5 Linien. Über eine Distanz von 1 Milliarde Spins spart dies 20 Milliarden interne Schleifeniterationen.
- **Einstieg ab der zweiten Walze:** Wenn Kombinationen strikt von links nach rechts ausgewertet werden und mindestens 3 Symbole erfordern, können wir die Prüfung ab der zweiten Walze (Index 1) starten. Wir schauen nach links auf Index 0. Wenn das Symbol links nicht übereinstimmt und kein Wild ist, bricht die Linie sofort ab (`continue / break`), was Operationen spart, bevor wir überhaupt den Rest der Linie betrachten.

```
// Konzept der ultraschnellen Linienprüfung für 3 Symbole (Positionen 0, 1, 2)
if ($flatReels[1] !== $flatReels[0]) {
    continue; // Linie abgebrochen, 1. und 2. Symbol stimmen nicht übereinstimmen
}
if ($flatReels[1] !== $flatReels[2]) {
    continue; // Linie abgebrochen, 2. und 3. Symbol stimmen nicht übereinstimmen
}
// Wenn hier erreicht – ein Gewinn mit 3 Symbolen ist bestätigt
```

Komplexe Wild-Logik wird durch eine flache Konfigurationskarte ersetzt — eine **Lookup-Tabelle (LUT)**, die tiefe Logikprüfungen in verzögerungsfreie Array-Schlüssel-Lookups verwandelt:

```
$wildMap = [
    'WILD_V1' => ['A' => true, 'B' => true, 'C' => true],
    'WILD_V2' => ['D' => true, 'E' => true],
];
```

Mikro-Optimierungen auf der PHP-Kernebene

Langsamer Ansatz	Hot Path Optimiert	Warum es funktioniert
<code>count(\$array)</code>	<code>\count(\$array)</code>	Der Aufruf aus dem globalen Namensraum (<code>\</code>) umgeht die Suche im aktuellen Namensraum.
<code>array_merge()</code> , <code>array_flip()</code>	Manuelle iterative <code>for</code> -Schleife	Integrierte Funktionen allokalieren für jedes Array neuen Speicher. Ein einfaches <code>for</code> kompiliert perfekt unter JIT.
Nutzung von <code>float</code> (0.05)	Nutzung von <code>int</code> (5)	Alle Berechnungen verwenden skalierte x100-Ganzzahlen (Cents). CPUs berechnen Ganzzahlen deutlich schneller und mit absoluter Präzision.
<code>random_int()</code>	<code>\mt_rand()</code>	Kryptografische Sicherheit ist hier überflüssig. Über eine große Distanz glättet das Gesetz der großen Zahlen Eigenheiten, während die Geschwindigkeit um ein Vielfaches steigt.

Kritisch wichtig: Kein xDebug in der Testumgebung aktiviert. Der Overhead für die Verfolgung von Ausführungsschritten zerstört die reine Rechenleistung vollständig.

Umgebungstrennung: Produktion vs. Emulation

In der **Produktion** benötigen Sie einen sicheren, überprüfbaren und zertifizierten RNG, der auf kryptografischen Primitiven basiert (z. B. `HMAC-SHA256`). Aber das ist ein massiver Geschwindigkeitskiller. Im **Emulationsmodus** (Monte Carlo) wird diese Logik komplett gegen Inline- `\mt_rand` ausgetauscht.

Die Emulation erfordert außerdem **Zero I/O**. Jegliches Schreiben von Protokollen, Interaktionen mit dem Dateisystem oder sogar ein einfaches `echo` in die Konsole drückt die Leistung auf den Boden. Die Datenverarbeitung läuft rein in Chargen (Batches), und Zustand Zusammenfassungen werden erst am Ende geschrieben.

4. Benchmark-Ergebnisse

Mit der neuen Kernarchitektur auf PHP 8.5 mit aktiviertem JIT stabilisierten sich die Leistungskennzahlen für unser 3x3-Spielfeld bei den folgenden Werten:

Metric	Value
Iterations	10 000 000
Iterations per Second	304 853.12
Total Execution time	00:00:32.802

Ergebnis: 10 Millionen Spins werden in nur 32 Sekunden auf einem einzigen Kern verarbeitet. Die reine Geschwindigkeit überschreitet 304.000 Spins/Sek. — eine 60-fache (6.000 %) Beschleunigung gegenüber dem standardmäßigen Legacy-Setup. Mathematische Verifizierungszyklen schrumpften von Wochen auf Minuten, was den Team-Output vervielfachte und die Kosten für die Cloud-Infrastruktur drastisch senkte.

5. Die Grenze der Vernunft: Dunkle Magie der Optimierung

Kann man PHP noch weiter antreiben? Ja, aber jeder Schritt ab diesem Punkt bricht die Wartbarkeit und das saubere Design:

- **Bitmasken:** Die Verarbeitung von Linienzuständen und Symbolübereinstimmungen über bitweise Operationen kann einen zusätzlichen Geschwindigkeitsschub von 25 % bis 50 % bringen, macht den Code jedoch unlesbar.
- **Totaler Prozeduraler Monolith:** Zusammenfassen von Klassen in eine einzige monolithische Datei, die rohe Arrays und Funktionen mit globalem Gültigkeitsbereich enthält.
- **Globaler Zustandsspeicher:** Umgehen der Datengenerierung durch Interaktion mit einem einzigen, vorallokierten Speicherlayout auf Anwendungsebene.
- **Precomputed Win Combo Maps:** Zuordnung aller möglichen Layouts zu Ergebnissen beim Start (z. B. `[A_A_A] => pay_multiplier`). Es verbraucht enorm viel RAM, verwandelt die Berechnung aber in eine einfache `isset()`-Suche. Sinnvoll für Emulationen, nutzlos für Produktions-HTTP-Zyklen.
- **SpIFixedArray:** Ein weit verbreiteter Mythos besagt, dass es immer die Leistung verbessert. Für flache Arrays mit einfachen Ganzzahlindizes führt es in PHP 8.5 zu Overhead im Vergleich zu standardmäßig optimierten Packed Arrays.

6. Migration zu Go (Golang) in 3 Wochen

Um die verbleibenden architektonischen Engpässe der PHP-Laufzeitumgebung zu beseitigen, habe ich die gesamte Kernlogik auf Go (Golang) portiert. Interessanterweise hatte ich noch nie zuvor in Go geschrieben; meine primäre Erfahrung lag in PHP und JS. Die Portierung dauerte ungefähr 2–3 Wochen Teilzeitarbeit, wobei KI-Engines nicht als Codegeneratoren, sondern als hochspezialisierte, strenge Suchfilter zur Beantwortung Tausender expliziter Fragen zu Speicherlayouts und Syntax behandelt wurden.

Die wichtigste Erkenntnis: PHP vs. Go

In PHP bleibt die größte Schwachstelle die dynamische Natur von Packed Arrays — eine geringfügige Indexverschiebung oder ein Typfehler fällt erst tief in den Simulationen auf. Go stoppt dies bereits zur Kompilierzeit:

- **Statische Typisierung als Schutzschild:** Der Compiler verhindert explizit das Laufzeittypchaos (z. B. den Versuch, ein `uint8` zu übergeben, wo ein expliziter Typ `SymbolID` erforderlich ist).
- **Performance auf Go:** Die Anwendung identischer Prinzipien — Data Locality, feste `[256]DataType` - Lookup-Tabellen, vorallokierte Slice-Kapazitäten und die Sicherstellung, dass Berechnungen strikt auf dem CPU-Stack ohne Heap-Allokationen ausgeführt werden (verifiziert über Escape-Analyse) — lieferte ****Millionen von Spins pro Sekunde auf einem einzigen Kern!****

7. Fazit

Die langsame Geschwindigkeit von PHP unter Rechenlasten ist ein Mythos, der von denjenigen aufrechterhalten wird, die die internen Abläufe nicht verstehen. Zu begreifen, wie die Engine mit Speicherlayouts (zvals) umgeht, wie JIT flache Ausführungspfade behandelt und wie CPUs Caching nutzen, ermöglicht es, Code mit echter **Mechanical Sympathy** für die Hardware zu schreiben.

Der Code verliert seine herkömmliche „Web-Schönheit“, gewinnt aber an echter technischer Eleganz, senkt die Infrastrukturkosten und beschleunigt die Feedbackschleifen in der Entwicklung, ohne die Produktionsqualität zu beeinträchtigen.